

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/362218896>

Leveraging Software Defined Perimeter (SDP), Software Defined Networking (SDN), and Virtualization to Build a Zero Trust Testbed with Limited Resources

Chapter · July 2022

CITATIONS

3

READS

1,811

1 author:



Dwight Horne

Baylor University

17 PUBLICATIONS 29 CITATIONS

SEE PROFILE

Leveraging Software Defined Perimeter (SDP), Software Defined Networking (SDN), and Virtualization to Build a Zero Trust Testbed with Limited Resources

Dwight Horne

AT&T Center for Virtualization, Southern Methodist University, Dallas, TX 75275, USA
rhorne@smu.edu

Abstract. Zero trust networks, zero trust protocol design, and zero trust software engineering are all active areas of research. Zero trust security also continues to proliferate in industry with many companies involved in one or more zero trust related projects and an Executive Order in the United States of America even mandating zero trust security with a near-term timeline for federal entities. The establishment of a zero trust testbed is a fundamental enabler to efficiently support both academic research and industry projects in the domain. In this paper, we describe how the zero trust features of a software defined perimeter (SDP) can be combined with the power and flexibility of software defined networking (SDN) and virtualization to build a zero trust testbed with limited resources. Even a student equipped with only a laptop can get started with zero trust experimentation today! We also outline useful tools for enhanced zero trust testbeds with additional (but still limited) resources and clearly show how SDP both aligns with key elements of zero trust architecture (ZTA) and contributes to satisfaction of core principles of Zero Trust by Design (ZTBD). This work will lead to additional enablers at ZeroTrustByDesign.com to facilitate design and implementation of zero trust testbeds in support of the continued evolution of zero trust research and practice. We further invite the community to join us in adding value to this growing body of zero trust knowledge and resources.

Keywords: zero trust testbed, software defined perimeter (SDP), software defined networking (SDN), zero trust architecture (ZTA), zero trust by design (ZTBD), zero trust security model

1 Introduction

Zero trust is everywhere. Take a quick tour through the exhibit hall at a modern security conference and one will undoubtedly encounter a number of zero trust focused marketing campaigns. Google proclaimed the benefits of zero trust in the commercial world when it revealed its zero trust centric BeyondCorp initiatives [1]. As for academia, at the time of this writing a quick search for “zero trust” on Google Scholar returned >3,000 results since 2018 (in less than 5 years). Yet a subset of both researchers and practitioners are still wondering when zero trust will really arrive in their local context, and what form it will take. Despite the broad range of zero trust documentation such as special publication standards [2], a zero trust maturity model [3], reference architectures [4][5], Executive Orders [6], and other guidance, the zero trust journey must transition at some point to hands-on experience. This is where having an adequate testbed for experimentation and testing becomes vital.

A zero trust testbed is foundational for both research and practice to enable elements of testing and experimentation along different dimensions such as security, usability, computational performance, network traffic characterization, compatibility, and more. Unfortunately, some may have the impression that zero trust testbeds are cost prohibitive, while others may be unsure where to begin if not with a specific vendor’s product lines. In this work, we demonstrate that a capable zero trust testbed can be achieved with very limited resources by combining the zero trust features of a software defined perimeter (SDP) with the flexibility of a software defined network (SDN) and virtualization. We also enumerate some more advanced options for building enhanced zero trust testbeds with additional (but still limited) resources.

The remainder of this paper is organized as follows. Section 2 provides background on ZTA, ZTBD, SDP, and SDN. Section 3 then describes construction of a zero trust testbed with limited resources and highlights tradeoffs when transitioning from light-weight virtualization to standard Type 1 or Type 2 virtualization options. Next, Section 4 provides a set of software tool options to enable enhanced zero trust testbed capabilities, while Section 5 provides concluding remarks.

2 Background

2.1 Zero Trust Architecture (ZTA) and Zero Trust by Design (ZTBD)

A zero trust security model abandons any notion of implicit trust. All users, devices, network flows, and resource access requests should instead be verified as trusted in the particular context prior to allowing access. As early as

2002, well before zero trust became a common part of the modern technology lexicon, researchers referred to a zero trust security model in designing a zero trust intrusion tolerant system that assumed compromise of the system was inevitable and periodically restored itself from a trusted backup [7]. But some underlying concepts behind zero trust undoubtedly predated that publication.

The zero trust model was later popularized as a rethinking of the network architecture to abandon the perimeter centric view of a trusted internal network in favor of a zero trust model [8]. After years of discussion and evolution, the zero trust architecture (ZTA) was more formally described in NIST Special Publication (SP) 800-207 [2]. Fig. 1 gives an overview of the main components of a ZTA as defined by NIST. There is a clear delineation between the control plane, where the Policy Decision Point (PDP) operates, and the data plane, where the Policy Enforcement Point (PEP) is informed by the PDP when governing the interactions of entities with protected resources. ZTA abandons the notion of a trusted internal network in favor of one that assumes hostility, grants no implicit trust, and strongly authenticates and authorizes all network flows and resource access requests.

Yet even with NIST SP 800-207, many other zero trust publications, and numerous vendors marketing zero trust products and services, it was recognized that a lack of consensus and some misunderstanding still existed. Moreover, most conversations focused on ZTA, but to fully realize the promise of the zero trust model, attention must also be placed in other areas such as zero trust software engineering and zero trust protocol design. Zero Trust by Design (ZTBD) was introduced to harmonize disparate zero trust guidance by distilling it down to fundamental principles to guide zero trust research and practice [9]. The foundational principles of ZTBD v1.0 appear in Fig 2. The ZTBD principles were further augmented with a set of good practices as well as zero trust patterns for reusable solutions to common challenges in a zero trust context. The goal is for the present work describing creation of zero trust testbeds with limited resources and related enablers to further supplement the growing body of knowledge at ZeroTrustByDesign.com to benefit zero trust research and practice.

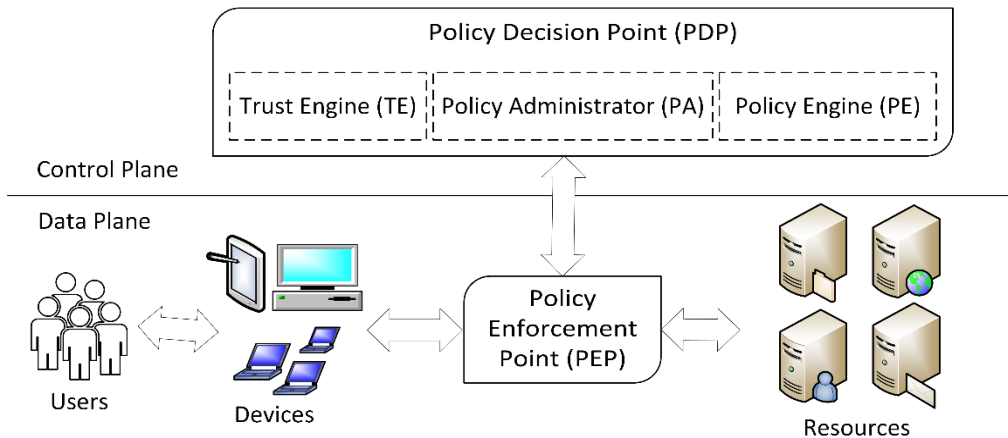


Fig. 1. Core concepts of zero trust architecture [2]

1. Trust no one (and no thing)
2. What perimeter?
3. Apply the principle of least privilege
4. Dynamic, risk-based policies
5. Require strong authentication and authorization
6. Log, monitor, inspect, and adapt
7. Employ defense in depth
8. Full lifecycle, zero trust security
9. Confidentiality and integrity by default
10. Strike the right balance with tradeoffs

Fig. 2. Foundational principles of Zero Trust by Design (ZTBD) v1.0 [9]

2.2 Software Defined Perimeter (SDP)

The concept of Software Defined Perimeter (SDP) flexibly provides an overlay network with dynamic trust provisioning and secure access, supporting protection of applications and services being accessed over an untrusted network. With SDP, resources are hidden from unauthorized parties until identity-centric trust has been established. This shift from a legacy mindset of static, perimeter focused security with trusted “internal” networks to a dynamically adaptive logical micro-perimeter requiring trust establishment aligns well with the zero trust security model and the principles of Zero Trust by Design (ZTBD) [9]. In fact, the revised and enhanced SDP 2.0

specification [10], published by the Cloud Security Alliance (CSA) in 2022, more clearly communicates that the elements of SDP directly support the principles of ZTA. Meanwhile, others had already recognized that the ability to provide perimeter-like security with trust-based access controls across any network with SDP were effective for implementing ZTA [11]. The SDP approach is also a great example of applying the *Single Packet Authorization* (SPA) pattern from ZTBD as a foundational enabler.

Fig. 3 reflects the key components of a notional SDP architecture and it is also annotated with corresponding mappings to the key concepts of a ZTA as defined by NIST [2]. The SDP Controller logically resides in the control plane, authenticating entities and authorizing access flows. In this way, the SDP Controller acts as the Policy Decision Point (PDP) in the NIST ZTA framework. The SDP IH1 and IH2 components of the diagram represent Initiating Hosts in SDP terminology, which are user devices or other entities that initiate connections in an SDP enabled environment. The Accepting Host (AH) entities are logical components that guard hidden services, allowing or disallowing access flows much like the Policy Enforcement Point (PEP) of the NIST ZTA framework. The AH can reside with the target resource as depicted with AH1 or it can serve as a physically separate SDP Gateway as depicted with AH2.

The SDP v2.0 specification describes a number of possible deployment models, outlines onboarding and access workflows, and delineates protocol details such as for SPA, mutual TLS authentication between components, and device validation. SDP directly aids satisfaction of core principles of ZTBD including principles 1, 2, 3, 5, 7, and 9. When part of a properly configured system, it can further contribute to the overall strategy for addressing ZTBD principles 4, 6, and 10. Consequently, the reference implementation of SDP from Waverly Labs is a good fit to serve as the cornerstone for building a zero trust testbed with limited resources.

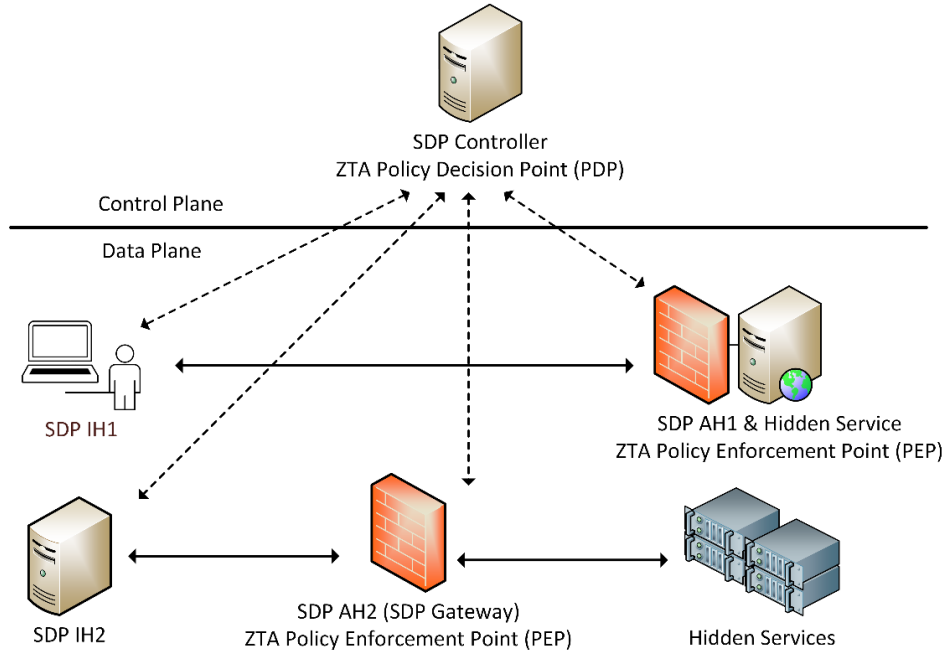


Fig. 3. Core ZTA components mapped to notional SDP architecture [2][10]

2.3 Software Defined Networking (SDN)

After McKeown et al. introduced OpenFlow [12], an open interface and protocol enabling access to the forwarding plane of networking equipment previously perceived as inflexible, Software Defined Networking (SDN) became one of the more popular topics in the field of data communications and OpenFlow rose to become a well-known enabler of SDN. With SDNs, the control plane is implemented in software and separated from the forwarding logic in the data plane. Notice the comparable notions of separate control and data planes across SDN, SDP, and ZTA, suggesting shared underlying design principles.

In the case of OpenFlow, which was standardized and is now managed by the Open Networking Foundation (ONF), an OpenFlow Controller can manage multiple OpenFlow forwarding devices. This architecture affords a number of optimization opportunities and the flexibility that comes with its programmatic nature [13]. But for the purposes of this paper, the focus is on how SDN can enable testing of large, complex network configurations with limited resources.

Network in a PC: Using Mininet for SDN Prototyping. The Mininet tool was introduced to facilitate rapid prototyping of large, complex network structures in resource constrained environments such as a typical laptop [14]. Mininet combines very lightweight virtualization with the versatility and power of SDNs to enable a plethora

of complex testing capabilities that would otherwise require significantly more resources. Complex network structures can be created from the Mininet command line interface (CLI) or programmatically via the application programming interface (API). While Mininet clearly has some limitations, it can serve as a key enabler for myriad zero trust test cases when researchers or practitioners are resource challenged.

3 Building a Zero Trust Testbed with Limited Resources

3.1 A Testbed in a Box for Zero Trust Network Simulation

Table 1 lists the recommended core software components of the zero trust testbed for limited resource environments along with a brief description of purpose and the licensing for each at the time of writing. The lightweight SDN virtualization of Mininet partnered with the resource hiding and authenticate/authorize-before-access model enforced by SDP, all further enabled by flexible open source licensing models, make them suitable for establishing a zero trust testbed for research or testing in practice prior to production deployments. Although the quickest way to get started is to use the available Mininet VM Image releases as a starting point, using a native Mininet installation (e.g., in Ubuntu) is recommended for maximum flexibility in customizing the testbed environment.

The recommended processors for the resource-limited zero trust testbed are relatively modern architectures (e.g., within the last decade) for virtualization support, sufficient instructions per second (IPS) single core performance, and core/thread counts for adequate parallel processing. This includes any 1st generation or later AMD Ryzen™ processors, 6th generation or later Intel® Core™ i3 and Core™ i5 processors, 3rd generation or later Intel® Core™ i7 processors, or any Core™ i9 processors. A large percentage of laptops and desktops sold within the last 5-7 years likely meet these processing requirements. Additionally, a minimum of 8 GB of RAM is also recommended, although testbeds with less may still be useful for basic test scenarios if running Ubuntu on bare metal.

Table 1. Core software components of a resource-limited zero trust testbed

Software	Purpose	License Type
Ubuntu ¹	Debian-based Linux host OS	Various
SDP Host/Gateway ²	fwknop derived SDP components	GPL v2
SDP Controller ³	SDP control module	GPL v3
Mininet ⁴	SDN enabled virtual network simulation	Mininet 2.3.1b1 License
Hidden Services	Protected services under test	Various

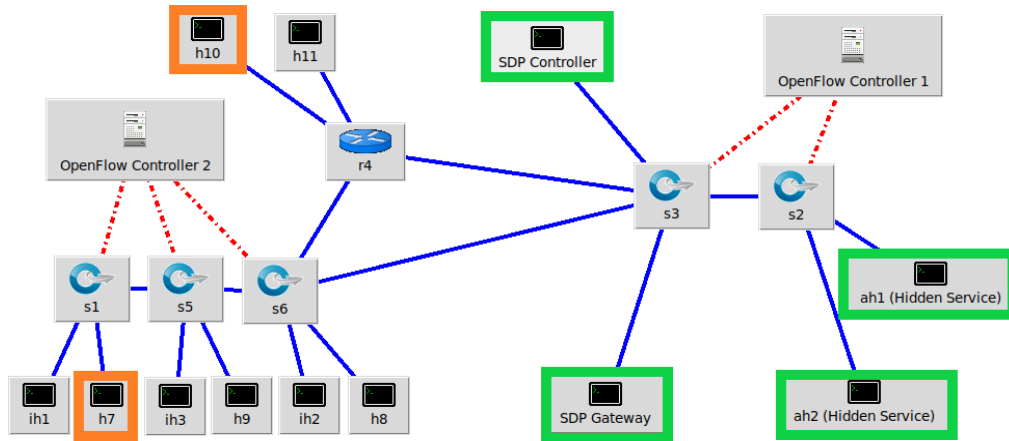


Fig. 4. Mininet network diagram for sample zero trust testbed with SDP components highlighted

The network layout of Fig. 4 represents a sample topology that one might create for a zero trust testbed with limited resources. The topology includes eight standard hosts connected to several OpenFlow and legacy switches/routers, an SDP Controller, an SDP Gateway, and two SDP Accepting Hosts for access to hidden services. For clarity, the Initiating Hosts are outlined with orange and the SDP components including the SDP

¹ <https://ubuntu.com/download>

² <https://github.com/WaverleyLabs/fwknop> | <https://www.waverleylabs.com/open-source-sdp/>

³ <https://github.com/WaverleyLabs/SDPcontroller> | <https://www.waverleylabs.com/open-source-sdp/>

⁴ <https://github.com/mininet/mininet> | <https://mininet.org/>

protected hidden resources are outlined with green. The example topology corresponds to approximately 95 lines of Python code to programmatically build the virtual network. The topology can be further refined by modifying the Python code, or via the Miniedit tool if a graphical user interface is preferred by the network designer for the testbed. Notice the power of the lightweight virtualization in Mininet and software-defined networking to enable prototyping and testing of foundations of ZTA with large or complex network configurations even using a testbed with very limited resources.

3.2 Virtualization Alternatives for Improved Fidelity of Testing

A number of studies have demonstrated the utility of Mininet as a virtual testbed for software defined networking and one recent study even showed that Mininet had adequate performance bounds in commodity servers to achieve aggregated bandwidths over 10 Gb/s with >1,000 hosts, up to 64-hop paths, and emulated network topologies with up to 64 different subnets [15]. However, there are also limitations to be aware of such as the complexity of properly securing the Mininet environment [16] and lesser isolation offered due to the lightweight virtualization approach. Consequently, a logical next step given increased resources for the zero trust testbed would be to transition from the lightweight virtualization of Mininet to heavier weight or full virtualization of a Type 1 (bare metal) or Type 2 (host-based) hypervisor environment.

Although the resource requirements would increase with this approach, the use of open source options for virtualization such as QEMU, the Linux Kernel-based Virtual Machine (KVM), VirtualBox, or the Xen hypervisor can still facilitate a cost effective testbed assuming adequate hardware. Engineers in academia or industry may have cost-effective commercial options as well depending on the context. The work of [16] provides an example of a virtualization based alternative to a Mininet testbed for SDN prototyping. A zero trust testbed enabled by a heavier weight virtualization option would be similar, but with key zero trust software components included such as SDP or those discussed with the enhanced testbed options of Section 4.

4 Building an Enhanced Zero Trust Testbed with More Resources

4.1 Additional Software Enablers for an Enhanced Testbed

The zero trust testbed can be further enhanced with additional tools. Table 2 gives name, purpose, and license type for tools recommended for advanced experimentation. Products like Pritunl Zero (open source BeyondCorp server) can be used instead of SDP in some use cases, while other tools support principles of ZTBD by providing features like multi-factor authentication (MFA), strong identity and access management (IAM), and more.

Table 2. Possible software components for more advanced zero trust testbed

Software	Purpose	License Type
Pritunl Zero ⁵	Open source BeyondCorp server, privileged access to web apps, SSH, “internal” services	Pritunl License (non-commercial use only)
OpenZiti ⁶	Open source zero trust networking for apps	Apache 2.0
Prometheus ⁷	ZT system/service monitoring, uses OpenZiti	Apache 2.0
Drools ⁸	Inference based rules engine	Apache 2.0
Mender ⁹	OTA updater for embedded devices, device config	Apache 2.0
OpenIAM ¹⁰	Identity & access management platform	Various
OpenAM CE ¹¹	Alternative identity & access management platform	CDDL 1.1
privacyIDEA ¹²	Open source (copyleft license) multi-factor authentication (e.g., OTP tokens, SMS, email, SSH keys, etc.)	Affero GPL v3
Gluu ¹³	IAM for web/mobile apps including MFA	Various
Trivy ¹⁴	Vulnerability scanner	Apache 2.0

⁵ <https://github.com/pritunl/pritunl-zero> | <https://zero.pritunl.com/>

⁶ <https://github.com/openziti/ziti> | <https://openziti.github.io/> | <https://ziti.dev/>

⁷ <https://github.com/openziti-test-kitchen/prometheus> | <https://prometheus.io/>

⁸ <https://github.com/kiegroup/drools> | <https://www.drools.org/>

⁹ <https://github.com/mendersoftware> | <https://mender.io/>

¹⁰ <https://github.com/OpenIAM> | <https://www.openiam.com/>

¹¹ <https://github.com/OpenIdentityPlatform/OpenAM> | <https://www.openidentityplatform.org/>

¹² <https://github.com/privacyidea/privacyidea> | <https://privacyidea.org/>

¹³ <https://github.com/GluuFederation> | <https://gluu.org> | <https://gluu.org/casa-self-service-mfa/>

¹⁴ <https://github.com/aquasecurity/trivy> | <https://www.aquasec.com/products/trivy/>

4.2 Building a Cloud-Enabled Zero Trust Testbed

Cloud-based software and services can offer additional zero trust testbed options but pricing models should be seriously considered. Some of the open source products in Tables 1 and 2 have commercial versions, resulting in a “try before you buy” opportunity. We refrain from mentioning specific vendors to avoid any perception of recommendation, sponsorship, or affiliation. But commercial product offerings of interest often use naming conventions or have descriptions related to zero trust network access (ZTNA), zero trust for the security service edge, cloud-native zero trust access, zero trust VPN alternative, zero trust software as a service (SaaS), or zero trust segmentation/micro-segmentation. Corporate employees should check to see what options might be available through their employer’s master service agreements with cloud providers. Many cloud-based service providers also offer free or low-cost options for students and academia to consider.

5 Conclusion and Future Work

The capabilities of SDP support implementation of the core components of ZTA and they facilitate alignment with core principles of ZTBD. Meanwhile, the power of SDN and light-weight virtualization can enable simulations with complex network topologies and large numbers of nodes. SDP and SDN can combine with other tools to create a zero trust testbed with very limited resources, supporting a level of zero trust testing and experimentation that might not otherwise be possible. In fact, using this approach a student equipped with only a laptop can start experimenting with zero trust today! Enhanced testbeds with additional (but still limited) resources and more zero trust tools can provide other advantages to researchers and practitioners. These enablers for fostering creation of zero trust testbeds will soon be added to ZeroTrustByDesign.com to join the growing repository of freely available zero trust knowledge and resources.

Lastly, this paper is not comprehensive and the zero trust landscape is evolving quickly. The zero trust knowledgeable reader will undoubtedly identify other beneficial software components of a zero trust testbed, whether with limited resources, significant local resources, or a cloud-based testbed. We extend another open invitation to the community to reach out to us at ZTBD@ZeroTrustByDesign.com with suggested additions as we work together to foster zero trust research and the next generation of the zero trust ecosystem.

References

1. Ward, R., and Beyer, B. (2014) Beyondcorp: a new approach to enterprise security, ;login, USENIX.
2. Rose, S., Borchert, O., Mitchell, S., Connelly, S. (2020) Zero trust architecture. NIST Special Publication (SP) 800-207. National Institute of Standards and Technology, U.S. Department of Commerce.
3. Cybersecurity and Infrastructure Security Agency (CISA), U.S. Department of Homeland Security (2021) Zero Trust Maturity Model – Pre-decisional Draft, Version 1.0.
4. Joint Defense Information Systems Agency (DISA) and National Security Agency (NSA) Zero Trust Engineering Team (2021) Department of Defense (DOD) Zero Trust Reference Architecture Version 1.0.
5. Cybersecurity and Infrastructure Security Agency, United States Digital Service, and Federal Risk and Authorization Management Program (2021) Cloud Security Technical Reference Architecture Version 1.0.
6. Executive Order No. 14028 (2021) Improving the Nation’s Cybersecurity. <https://www.gsa.gov/technology/technology-products-services/it-security/executive-order-14028-improving-the-nations-cybersecurity/>. Accessed 20 March 2022.
7. Sood, A.K., Huang, Y., Simon, R., White, E., and Cleary, K. (2002) Zero trust intrusion containment for telemedicine, George Mason University, Fairfax, VA, USA.
8. Kindervag, J. (2010) Build security into your network's DNA: The zero trust network architecture. Forrester Research Inc., pp 1-26.
9. Horne, D. and Nair, S. (2021) Introducing Zero Trust by Design: Principles and Practice Beyond the Zero Trust Hype. In: Daimi, K., Arabnia, H., Deligiannidis, L., Tinetti, F. (ed) Advances in Security, Networks, and Internet of Things (SAM, ICWN, ICOMP, ESCS 2021), pp 512-525, Springer Cham.
10. Cloud Security Alliance (CSA) (2022) Software-Defined Perimeter (SDP) Specification v2.0. <https://cloudsecurityalliance.org/artifacts/software-defined-perimeter-zero-trust-specification-v2/>. Accessed 15 March 2022.
11. Lefebvre, M., Nair, S., Engels, D.W. and Horne, D. (2021) Building a Software Defined Perimeter (SDP) for Network Introspection. In *2021 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, pp 91-95, IEEE.
12. McKeown, N., Anderson, T., Balakrishnan, H., Parulkar, G., Peterson, L., Rexford, J., Shenker, S. and Turner, J. (2008) OpenFlow: enabling innovation in campus networks. *ACM SIGCOMM computer communication review*, 38(2), pp 69-74, ACM.
13. Lara, A., Kolasani, A. and Ramamurthy, B. (2013) Network innovation using openflow: A survey. *IEEE communications surveys & tutorials*, 16(1), pp 493-512, IEEE.
14. Lantz, B., Heller, B., and McKeown, N. (2010) A network in a laptop: rapid prototyping for software-defined networks. In: Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks, pp 1-6, ACM.
15. Muelas, D., Ramos, J. and Lopez de Vergara, J. E. (2018) Assessing the limits of mininet-based environments for network experimentation. *IEEE Network* 32(6): pp 168-176, IEEE.
16. Flauzac, O., Gallegos Robledo, E. M., and Nolot, Florent. (2019) Is Mininet the right solution for an SDN testbed? In: 2019 IEEE Global Communications Conference (GLOBECOM), pp 1-6, IEEE.